
API Documentation

chtting-relay — an OpenAI-compatible inference relay

Wire protocol	OpenAI Chat Completions (HTTP + SSE)
Base URL	<code>https://<your-relay-host>/v1</code>
Auth	<code>Authorization: Bearer <client key></code>
Audience	Integrators connecting a client or a service to this relay

Everything in this document describes the relay's own behaviour: the request it accepts, the response it returns, and the fields it guarantees. The model that actually produces the text sits behind the relay and is never named in any response — treat the model ids in `GET /v1/models` as the only model identity that exists.

Contents

1. Base URL and endpoints 3
2. Authentication 3
3. Making a request 4
4. User ID and KV-cache isolation 6
5. The response we send back 7
6. Streaming (SSE) 9
7. Listing models: /v1/models and /models 11
8. Errors 14
9. Limits and operational notes 15
10. Copy-paste examples 16

1. Base URL and endpoints

The relay speaks the OpenAI HTTP API. Any OpenAI-compatible SDK, client or agent framework works unchanged — point its `base_url` at the relay and give it a client key.

```
base_url = https://<your-relay-host>/v1
```

Every path exists with and without the `/v1` prefix. Some clients append `/v1` themselves, others are handed a base URL that already ends in it. Both spellings hit the same handler, so neither configuration is wrong.

Method	Path	Key?	What it does
GET	/health	no	Liveness, version, how many models and backends are up, requests in flight.
GET	/v1/models /models	yes	The model catalogue. See section 7.
GET	/v1/models/{id} /models/{id}	yes	One model's entry, or <code>404 model_not_found</code> .
POST	/v1/chat/completions /chat/completions	yes	Chat, streaming and non-streaming. The main endpoint.
POST	/v1/completions /completions	yes	Legacy text completions. Same routing, same billing.
POST	/v1/embeddings /embeddings	yes	Embeddings, when the routed model's backend offers them.
OPTIONS	any of the above	no	CORS preflight.

GET `/health` is the only endpoint that never needs a key, and the only one safe to poll from a monitor:

```
{ "status": "ok", "service": "chtting-relay", "version": "2.0.0",  
  "models": 3, "backends": 1, "uptime_s": 81422, "in_flight": 2 }
```

Tunnel addresses move. If the relay is published through a quick Cloudflare tunnel, the hostname changes every time the tunnel restarts. Ask the operator for a named tunnel hostname before hard-coding a URL anywhere.

2. Authentication

Every endpoint except `/health` and CORS preflight takes a client key as a bearer token:

```
Authorization: Bearer Kunci-Zeiko-XXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

A key sent bare — the value with no `Bearer` scheme in front of it — is also accepted, because some clients send it that way.

The model listing needs a key too. `GET /v1/models` is authenticated on this relay. A client that fetches the model list before it has been given credentials will get `401`, not an empty list. Configure the key first, then discover models.

The two kinds of client key

A key has a kind, and the kind decides one thing: **whose behalf this request is on**. Everything else follows from that. Your operator will tell you which kind you hold.

	company	private
Who is behind the key	many end users	one holder
Identity sent upstream	the one you send (see section 4)	the key's own identity
If you send <code>user</code>	used	ignored
Usage broken down by	end user	key
Output speed cap	may apply	never applies

A **company** key is a reseller key: each call carries its own end user, and that id is what isolates one customer's prompt cache from another's. A **private** key is one person, so anything you put in `user` is ignored and the key's own identity (a stable fingerprint, not the key itself) is used instead.

3. Making a request

`POST /v1/chat/completions` takes the ordinary OpenAI request body. Two fields are required: `model` and `messages`.

```
curl -N https://<your-relay-host>/v1/chat/completions \
-H "Authorization: Bearer Kunci-Zeiko-XXXXXXXXXXXXXXXXXXXXXXXXXXXX" \
-H "Content-Type: application/json" \
-H "X-User-Id: customer-42" \
-d '{
  "model": "Wissangeni-512B-V1",
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Hello!"}
  ],
  "stream": true,
  "temperature": 0.7,
  "max_tokens": 512,
  "user": "customer-42"
}'
```

What the relay acts on, and what it forwards

Parameters the relay does not act on are forwarded to the backend untouched. Whether the second group has any effect therefore depends on the routed model, not on this relay.

Parameter	Handled by	Notes
<code>model</code>	relay	Required. One of the ids from <code>/v1/models</code> , or one of that model's aliases. Matched exactly and case-sensitively. Mapped internally to the backend's own model name, which is never disclosed.

Parameter	Handled by	Notes
<code>messages</code>	relay	Required. A system prompt may be prepended, appended or merged into yours, depending on the model.
<code>stream</code>	relay	<code>true</code> returns SSE, <code>false</code> a single JSON body. Every model supports both.
<code>max_tokens</code> <code>max_completion_tokens</code>	relay	Clamped down to the model's ceiling, never refused for being too large. Whichever of the two field names you send is the one kept. Omit them and the model's ceiling is applied.
<code>stop</code>	relay	Up to four entries, as OpenAI. The relay may add one of its own within that limit.
<code>user</code> / <code>user_id</code>	relay	The prompt-cache isolation key. See section 4 — read it.
<code>reasoning_effort</code>	relay	<code>none</code> , <code>minimal</code> , <code>low</code> , <code>medium</code> , <code>high</code> , <code>max</code> . Selects the system prompt and can move the price. Also read from <code>reasoning.effort</code> , <code>thinking.effort</code> , <code>extra_body.reasoning.effort</code> , the on/off switches <code>enable_thinking</code> , <code>reasoning.enabled</code> , <code>thinking.enabled</code> , <code>thinking.type: "disabled"</code> , and from a budget in <code>thinking.budget_tokens</code> , <code>reasoning.max_tokens</code> or <code>thinking_budget</code> .
<code>reasoning.exclude</code> <code>include_reasoning</code>	relay	Ask for the reasoning trace to be left out of the reply. Either can only remove a trace; neither can turn on one the model keeps to itself.
<code>stream_options</code>	relay	Replaced by the relay's own. You never need <code>include_usage</code> : usage always arrives on the final chunk.
<code>usage</code> , <code>route</code> , <code>models</code> , <code>transforms</code> , <code>provider</code> , <code>plugins</code> , <code>preset</code>	relay	OpenRouter's routing vocabulary. Accepted so a client that always sends them is not rejected, then dropped — routing is already settled by the time the body is built, and a strict backend answers 400 to fields it does not recognise.
<code>temperature</code> , <code>top_p</code> , <code>top_k</code> , <code>presence_penalty</code> , <code>frequency_penalty</code> , <code>repetition_penalty</code> , <code>min_p</code> , <code>seed</code> , <code>logit_bias</code> , <code>logprobs</code> , <code>top_logprobs</code>	backend	Forwarded as sent.
<code>response_format</code> , <code>tools</code> , <code>tool_choice</code>	backend	Forwarded as sent. Whether JSON mode or tool calling works is the backend model's business.

Check `supported_parameters` before you rely on a parameter. Each model's entry in `/v1/models` lists exactly what it accepts, and that list is derived from the live configuration — a parameter an operator has configured the relay to drop disappears from it. Sending something that is not listed is not an error; it simply has no effect.

Requests to `/v1/completions` and `/v1/embeddings`

`/v1/completions` takes the legacy body and goes through the same routing, the same quotas and the same accounting as chat. `/v1/embeddings` takes `model` plus `input` (a string or an array of strings) and returns the provider's embedding response with one change: `model` in the response is rewritten back to the public id you asked for. There is no streaming and no reshaping on that path.

4. User ID and KV-cache isolation

Send a stable, opaque identifier for your end user on every request. This is the single most important thing to get right in an integration, and it is worth a section of its own.

Why: KV-cache isolation

The backends behind this relay cache the attention keys and values computed for a prompt prefix (the KV cache) and key that cache per user. The user id you send is what partitions it.

- **Correctness.** Without an id, every caller behind one API key lands in the same cache partition. One tenant's prompt prefix becomes reusable by another tenant's request — exactly the leak the id exists to prevent.
- **Cost and latency.** A cache hit is billed at the cache-read rate, which is a fraction of the fresh-input rate, and it removes most of the time-to-first-token on a long system prompt. A user id that changes between requests destroys every hit; a stable one keeps them.

Where to put it

Two places, and the body wins over the headers:

Where	Field / header	Precedence
Request body	<code>user</code>	1 — checked first
Request body	<code>user_id</code>	2 — used when <code>user</code> is absent or blank
Header	<code>X-User-Id</code>	3
Header	<code>X-User</code>	4
Header	<code>X-OpenAI-User</code>	5
Header	<code>X-KV-User</code>	6

The first of those that carries a non-empty value (after trimming whitespace) wins. Using the body field `user` is recommended: every OpenAI SDK already exposes it, so nothing custom is needed.

```
// OpenAI SDK, any language – the standard field is enough
{
  "model": "Wissangeni-512B-V1",
  "messages": [ ... ],
  "user": "tenant-42:user-9f3c1ab7"
}
```

```
# Or as a header, for clients that cannot add a body field
X-User-Id: tenant-42:user-9f3c1ab7
```

What we do with it

Whichever way it arrives, the same string is recorded on the request row, written to the log line, and **forwarded to the backend in three places at once** so that no backend spelling is missed:

Sent upstream as	Why
Header <code>X-User-Id</code>	The header name is configurable per backend; <code>x-user-id</code> is the default. Header values must be ASCII, so an id that is not ASCII is simply not sent as a header — the body fields below still carry it.

Sent upstream as	Why
Body <code>user</code>	OpenAI's spelling.
Body <code>user_id</code>	The second spelling, because backends disagree on which one they read. A backend reading only the one we did not send would pool every caller behind this relay into a single cache — which is the whole failure mode.

If you send nothing, that is allowed: the request is recorded with an empty user, and `user` and `user_id` are removed from the body before it goes upstream rather than being left at whatever unusable value arrived. Cache isolation is then whatever the backend does by default, which is to say: none of your own.

Rules for choosing the value

- **Stable.** The same end user must get the same string on every request, across sessions and across process restarts. This is what makes the cache hit.
- **Unique per end user.** Two different people must never share one id, or they share a cache partition.
- **Opaque.** Send a pseudonymous identifier — a hash, a UUID, an internal account id. **Do not send an email address, a phone number, a username or a real name.** The id travels to a third-party backend and lands in its logs.
- **Short.** The recorded value is truncated at 120 characters. Keep it well under that.
- **ASCII.** Non-ASCII ids still work but cannot ride in the header, so the backend only sees them in the body.

Private keys: if you hold a `private` key, none of the above applies — whatever you send is ignored, and the key's own stable fingerprint is what travels upstream and what your usage is filed under. You cannot land in another holder's cache partition, and nobody can land in yours.

Cache retention is the backend's to state, not this relay's. Nothing about your prompt is stored here beyond a truncated preview on the operator's own Requests screen. If you need a retention figure in writing, ask the operator to obtain it from the upstream provider.

5. The response we send back

Nothing the backend says about itself reaches you. Every reply is **rebuilt** from an envelope the relay owns — its own id, its own timestamp, its own model name — and only a named handful of fields are copied across. The backend's model name, host, request id and error wording never appear in any response.

Non-streaming body

```
{
  "id": "32dadfde-f3dd-4ad8-acc5-387b72415f46",
  "object": "chat.completion",
  "created": 1789936200,
  "model": "Wissangeni-512B-V1",
  "provider": "zeikoai",
  "choices": [
    {
      "index": 0,
      "message": {
```

```

    "role": "assistant",
    "content": "Hello! How can I help you today?",
    "reasoning_content": "The user greeted me...",
    "tool_calls": []
  },
  "finish_reason": "stop",
  "native_finish_reason": "stop"
}
],
"usage": {
  "prompt_tokens": 18,
  "completion_tokens": 256,
  "total_tokens": 274,
  "prompt_tokens_details": { "cached_tokens": 12 },
  "completion_tokens_details": { "reasoning_tokens": 64 },
  "usage": 0.102949871,
  "cost": 0.102949871
}
}

```

Field	Type	Meaning
<code>id</code>	string	A UUIDv4 minted by this relay when the request arrived. Unrelated to any id the backend used. It is the same value as the <code>X-Relay-Request-Id</code> response header and the same value on every frame of a stream. Quote it in any support request — the operator can pull the exact row.
<code>object</code>	string	<code>"chat.completion"</code> , or <code>"chat.completion.chunk"</code> on a stream frame.
<code>created</code>	integer	Unix seconds, stamped by the relay when the request arrived.
<code>model</code>	string	Always the public id you asked for, never the backend's own name.
<code>provider</code>	string	The relay's provider slug. Present unless the operator blanked it.
<code>choices[].index</code>	integer	Choice index, <code>0</code> for a single completion.
<code>choices[].message.role</code>	string	Always <code>"assistant"</code> .
<code>choices[].message.content</code>	string	The answer. May carry a configured prefix or suffix, and text rewrite rules may have been applied.
<code>choices[].message.reasoning_content</code>	string	The thinking trace, when the model exposes one and the route passes it through. Depending on how the route is configured it may instead appear as <code>reasoning</code> , be folded into <code>content</code> between tags, or be stripped entirely. Absent when empty.
<code>choices[].message.tool_calls</code>	array	Assembled tool calls, each <code>{id, type:"function", function:{name, arguments}}</code> . Absent when there are none.
<code>choices[].finish_reason</code>	string	<code>stop</code> , <code>length</code> , <code>tool_calls</code> , etc.
<code>choices[].native_finish_reason</code>	string	The same value under OpenRouter's spelling, for clients that read it.

The usage block — the fields to bill from

`prompt_tokens` is counted by **this relay's own tokenizer over your own messages**, measured before anything was injected. A system prompt the relay adds on your behalf is not charged to you, so the figure matches what you actually sent.

Field	Type	Meaning
<code>usage.prompt_tokens</code>	integer	Input tokens, over your own messages only.
<code>usage.completion_tokens</code>	integer	Output tokens, reasoning tokens included.
<code>usage.total_tokens</code>	integer	The two above, added.
<code>usage.prompt_tokens_details.cached_tokens</code>	integer	The cache-hit portion of the input. Never larger than <code>prompt_tokens</code> . Absent when zero — this is your signal that user-id isolation is working.
<code>usage.completion_tokens_details.reasoning_tokens</code>	integer	The reasoning portion of the output. Absent when zero.
<code>usage.usage</code>	number	What this request cost, in USD, to nine decimal places. Absent only for a model nobody has priced at all — never <code>0</code> to mean "free".
<code>usage.cost</code>	number	The same number under OpenRouter's spelling. It is never a second, different price.

Nine decimal places is deliberate. A short request at a tenth of a dollar per million tokens comes to a few millionths of a cent; rounding to six places would report every small request as free. Parse it as a decimal, not as a float, if you are reconciling invoices.

Response headers

Header	Value
<code>X-Relay-Request-Id</code>	The same UUIDv4 as <code>id</code> in the body. Present on both streaming and non-streaming replies. Log it.
<code>Content-Type</code>	<code>application/json</code> , or <code>text/event-stream; charset=utf-8</code> when streaming.
<code>Retry-After</code>	Seconds to wait. Sent with <code>429</code> (per-minute limit) and <code>503</code> (overloaded).
<code>Access-Control-Allow-Origin</code>	Per the relay's CORS policy, when the origin is allowed.

6. Streaming (SSE)

Send `"stream": true` and the response is `text/event-stream`: one `data:` frame per chunk, then a final frame carrying usage, then `[DONE]`.

Response headers on a stream

```
content-type:    text/event-stream; charset=utf-8
cache-control:   no-cache, no-transform
connection:      keep-alive
x-accel-buffering: no
x-relay-request-id: 32dadfde-f3dd-4ad8-acc5-387b72415f46
```

The frame sequence

```
data: {"id":"32dadfde...", "object":"chat.completion.chunk", "created":1789936200,
      "model":"Wissangeni-512B-V1", "provider":"zeikoai",
      "choices":[{"index":0, "delta":{"role":"assistant"},
        "finish_reason":null, "native_finish_reason":null}]}

data: {"id":"32dadfde...", "object":"chat.completion.chunk", "created":1789936200,
      "model":"Wissangeni-512B-V1", "provider":"zeikoai",
      "choices":[{"index":0, "delta":{"content":"Hel"},
        "finish_reason":null, "native_finish_reason":null}]}

: Zeiko is still here, Just be patience

data: {"id":"32dadfde...", "object":"chat.completion.chunk", "created":1789936200,
      "model":"Wissangeni-512B-V1", "provider":"zeikoai",
      "choices":[{"index":0, "delta":{"content":"lo!"},
        "finish_reason":"stop", "native_finish_reason":"stop"}]}

data: {"id":"32dadfde...", "object":"chat.completion.chunk", "created":1789936200,
      "model":"Wissangeni-512B-V1", "provider":"zeikoai",
      "choices":[],
      "usage":{"prompt_tokens":18, "completion_tokens":256, "total_tokens":274,
        "usage":0.010293, "cost":0.010293}}

data: [DONE]
```

(Frames are shown wrapped for print. On the wire each `data:` payload is a single line, and every frame is terminated by a blank line.)

What a client must handle

Frame	Rule
First frame	Carries <code>delta.role: "assistant"</code> and usually no content. Do not assume it has text.
Content frames	<code>delta.content</code> is a string fragment. Concatenate in arrival order. A reasoning trace, when the route passes one through, arrives as <code>delta.reasoning_content</code> or <code>delta.reasoning</code> alongside — or folded into <code>content</code> between tags, depending on the route's configuration.
Tool calls	<code>delta.tool_calls</code> arrives as partial fragments keyed by <code>index</code> , in the usual OpenAI way: accumulate <code>function.arguments</code> across frames before parsing.
Comment lines	A line beginning with <code>:</code> is an SSE comment — a keep-alive, sent after roughly 15 seconds of backend silence (the operator can change the interval and the text, or switch it off). Discard it. A compliant SSE parser already does; a hand-rolled one that does <code>line.startsWith("data: ")</code> is also fine. Never try to JSON-parse it. It exists to hold the connection open through tunnels and NATs, and it is the relay's own — the backend's keep-alives are parsed and dropped here, never forwarded.
Final usage frame	Has an empty <code>choices</code> array and carries <code>usage</code> . It arrives whether or not you sent <code>stream_options.include_usage</code> . A client that assumes <code>choices[0]</code> exists on every frame will crash here — guard it.
Error frame	If the backend fails part-way through, a frame carrying <code>{"error": {"message": "...", "type": "upstream_error"}}</code> is emitted before <code>[DONE]</code> , so a stream that has already started never just ends in silence. The HTTP status stays <code>200</code> : it was already sent.
<code>data: [DONE]</code>	The terminator. Always last, and not JSON.

Guarantees and gotchas

- `id`, `created`, `model` and `provider` are identical on every frame of one response.
- Frames may be split across TCP reads. Buffer until a blank line before parsing, and accept both `\n\n` and `\r\n\r\n` as the separator.
- Multi-byte characters are reassembled by the relay, so a UTF-8 character is never split across two frames' `delta.content`.
- Output may be **paced**: a company key's stream can be held to a configured tokens-per-second ceiling, so a slow stream is not necessarily a slow model. Private keys are never paced.
- Use `curl -N`, and turn off response buffering in your HTTP client. The relay sends `x-accel-buffering: no` so nginx and cloudflared do not buffer; an intermediary you control might still.
- Set your client timeout above the backend timeout (default 10 minutes) — with keep-alives on, an idle-read timeout will not fire spuriously, but a total-request timeout can.
- If you disconnect mid-stream the relay stops and records the request as cancelled. You are still accounted for what was generated up to that point.

7. Listing models: `/v1/models` and `/models`

Both paths return the identical document. `/v1/models/{id}` and `/models/{id}` return a single entry from it.

```
curl https://<your-relay-host>/v1/models \
-H "Authorization: Bearer Kunci-Zeiko-XXXXXXXXXXXXXXXXXXXXXXXXXXXX"
```

What you need to know before you call it

1. **It requires a client key.** Unauthenticated, it answers `401` with `invalid api key`. Several tools fetch the model list at startup before credentials are configured; that is the usual cause of a mysterious `401` on an otherwise working setup.
2. **Only enabled models appear.** A model absent from the list is switched off, not broken. The list reflects the live configuration and can change without warning when the operator edits it — re-fetch periodically rather than caching it for the life of a process.
3. **The ids in it are the only values `model` accepts** — plus each model's aliases, which are not published in the listing. Matching is exact and case-sensitive.
4. **`/v1/models/{id}` matches the listed `id` only.** An alias that works fine in a chat request returns `404` here. Ids containing a slash (`vendor/name`) work as written — the path accepts the whole remainder.
5. **The backend's real model name is never in the document**, anywhere, at any nesting level. Do not try to infer it; route on the public id.
6. **Prices are decimal strings in USD per single token** — not per million, and not JSON numbers. `"0.000000003"` loses its last digits through a float, so parse it as a decimal if you are reconciling a bill.
7. **A missing `pricing` object means nobody priced that model**, not that it is free. A price that was never set is published as nothing rather than as zero.

The document shape

OpenAI's envelope on the outside, so an unmodified OpenAI client unwraps it; OpenRouter's model document on the inside, so a client that wants to know what a model costs and accepts can read it without asking anyone.

```
{
  "object": "list",
  "data": [
    {
      "id": "Wissangeni-512B-V1",
      "canonical_slug": "Wissangeni-512B-V1",
      "hugging_face_id": null,
      "name": "Wissangeni 512B V1",
      "created": 1789900000,
      "description": "A sparse mixture-of-experts model.",
      "context_length": 128000,
      "architecture": {
        "modality": "text->text",
        "input_modalities": ["text"],
        "output_modalities": ["text"],
        "tokenizer": "o200k_base",
        "instruct_type": null
      },
      "pricing": {
        "prompt": "0.00000015",
        "completion": "0.0000006",
        "input_cache_read": "0.000000003",
        "internal_reasoning": "0.0000006",
        "request": "0",
        "overrides": [
          { "utc_days": ["monday","friday"], "utc_start": 100, "utc_end": 400,
            "prompt": "0.0000003", "completion": "0.0000012",
            "input_cache_read": "0.00000006" }
        ]
      },
      "top_provider": {
        "context_length": 128000,
        "max_completion_tokens": 8192,
        "is_moderated": false
      },
      "per_request_limits": null,
      "supported_parameters": ["frequency_penalty","max_tokens","reasoning",
                             "reasoning_effort","response_format","seed","stop",
                             "temperature","tool_choice","tools","top_p", "..."],
      "default_parameters": { "temperature": 0.7 },
      "supported_voices": null,
      "knowledge_cutoff": null,
      "expiration_date": null,
      "links": {},
      "reasoning": {
        "mandatory": false,
        "default_enabled": true,
        "supported_efforts": ["max","high","medium","low","none"],
        "default_effort": "high"
      },
      "object": "model",
      "owned_by": "ZeikoAI",
      "display_name": "Wissangeni 512B V1"
    }
  ]
}
```

Field reference

Field	Meaning
<code>id</code>	The value to put in <code>model</code> . The one identifier that matters.
<code>canonical_slug</code>	The dated name of this exact snapshot, beside the moving <code>id</code> . Pin this one to pin a behaviour.
<code>name</code> / <code>display_name</code>	Human-readable label. Falls back to <code>id</code> .
<code>created</code>	Unix seconds.
<code>context_length</code>	Total context window in tokens. A prompt over the model's input ceiling is refused with <code>413</code> , not truncated.
<code>architecture.modality</code>	<code>"text->text"</code> and similar, built from the two modality arrays beside it.
<code>architecture.tokenizer</code>	The tokenizer family the relay counts this model with — the same one that produces <code>usage.prompt_tokens</code> . Use it if you want to pre-count locally.
<code>pricing.prompt</code>	Fresh input, USD per token, as a decimal string.
<code>pricing.completion</code>	Output, USD per token.
<code>pricing.input_cache_read</code>	Cache-hit input. Absent means a cache hit is not discounted, not that it is free.
<code>pricing.input_cache_write</code>	Cache write, when the provider charges for one.
<code>pricing.internal_reasoning</code>	Reasoning tokens. Absent means they are billed at the output rate.
<code>pricing.request</code>	A flat per-request amount, when there is one.
<code>pricing.overrides[]</code>	Time-of-day pricing. Each entry has optional <code>utc_days</code> (lowercase weekday names) and optional <code>utc_start</code> / <code>utc_end</code> as <code>HHMM</code> integers in UTC — <code>100</code> is 01:00, <code>1430</code> is 14:30. The window is half-open: start inclusive, end exclusive, and a window whose end is before its start wraps midnight. No times means all day. Every override publishes a complete price , not just the fields it moved, so you never have to walk back up the list.
<code>top_provider.max_completion_tokens</code>	The output ceiling. A larger <code>max_tokens</code> is clamped to this, not refused.
<code>per_request_limits</code>	Always <code>null</code> here.
<code>supported_parameters</code>	Derived from the live configuration, sorted, deduplicated. A parameter the relay is configured to drop is not advertised.
<code>default_parameters</code>	Defaults the relay applies when you do not send them. You can override any of these.
<code>reasoning</code>	Present only on models that reason . <code>supported_efforts</code> is the menu <code>reasoning_effort</code> accepts; <code>default_effort</code> is what you get by naming none. Its absence is how you detect a non-reasoning model.
<code>owned_by</code>	The relay's own owner label, never the upstream vendor.
<code>hugging_face_id</code> , <code>knowledge_cutoff</code> , <code>expiration_date</code> , <code>supported_voices</code>	<code>null</code> unless the operator filled them in.

Reconciling a bill from the listing

The listing is a promise, and it is kept: the `usage.usage` figure on a response is the published rates multiplied by the token counts on the same response. Cached input is carved out of `prompt_tokens` and reasoning out of `completion_tokens`, so nothing is counted twice:

```
cost = (prompt_tokens - cached_tokens) * pricing.prompt
      + cached_tokens * pricing.input_cache_read
      + (completion_tokens - reasoning_tokens) * pricing.completion
      + reasoning_tokens * pricing.internal_reasoning
      + pricing.request
```

Use the `overrides` entry that covers the UTC moment the request ran, if any, and the standing rates otherwise.

`/provider/models` is a separate document, meant for an aggregator polling this relay as an upstream provider. It is off by default and may require its own token. It is not part of the OpenAI surface — do not point a client at it.

8. Errors

Failures come back in the OpenAI error shape, with the HTTP status matching.

```
{
  "error": {
    "message": "model \"gpt-9\" is not available on this relay",
    "type": "invalid_request_error",
    "code": "model_not_found"
  }
}
```

Status	code	When
400	—	Malformed JSON, a missing or empty <code>model</code> field, or a body this model will not accept.
401	<code>invalid_api_key</code>	Missing or unknown client key. Also what an unauthenticated <code>GET /v1/models</code> gets.
403	<code>invalid_api_key</code>	The key is disabled.
403	<code>model_forbidden</code>	The key exists but is not allowed that model.
403	<code>forbidden</code>	The calling IP is blocked.
404	<code>model_not_found</code>	Unknown model id — or a known alias asked for at <code>/v1/models/{id}</code> .
404	<code>not_found</code>	No route for that path.
413	<code>too_large</code>	The body is over the size limit, or the prompt is over the model's input ceiling. The message says which.
429	<code>rate_limit_exceeded</code>	A per-minute rate limit or a daily request/token quota is spent. The per-minute refusal carries <code>Retry-After</code> .
499	—	Recorded, not sent: you disconnected before the answer finished.
502	<code>upstream_unavailable</code>	The model could not answer. Retry with backoff; if it persists, tell the operator and quote <code>X-Relay-Request-Id</code> .

Status	code	When
503	overloaded	Too many requests in flight and the queue is full or timed out. Carries <code>Retry-After</code> .
504	timeout	The model took too long to answer.

An upstream failure is reported as this relay's failure, on purpose. The status is mapped by class and the wording is ours: no upstream name, host, model or error text ever reaches a caller. In particular, an upstream rejection of the relay's own credentials comes back as `502`, never as `401` — because your key was fine, and telling you otherwise would send you debugging the wrong thing.

Retry policy we recommend

- `429` and `503`: honour `Retry-After` when present, otherwise exponential backoff from ~ 1 s.
- `502` and `504`: retry up to two or three times with backoff. The relay already retries transient upstream failures and falls through to fallback backends before it gives up, so a `502` that reaches you has already been tried more than once.
- `400`, `401`, `403`, `404`, `413`: do not retry. Fix the request or the credentials.

9. Limits and operational notes

Exact values are per-relay and per-key; ask your operator for yours. The defaults are listed so you know what shape of limit to expect.

Limit	Default	Behaviour when exceeded
Request body size	20 MB	<code>413</code> before the body is read.
Requests in flight	512	Past this, requests queue rather than fail.
Queue	2048 waiting, 30 s wait	A full or timed-out queue answers <code>503</code> with <code>Retry-After</code> , rather than holding you for a slot you will not reach.
Requests per minute	per key	<code>429</code> with <code>Retry-After</code> .
Requests per day	per key	<code>429</code> . Resets on the relay's local calendar day.
Tokens per day	per key	<code>429</code> . Input and output counted together.
Max input tokens	per model	<code>413</code> . The prompt is refused, never truncated.
Max output tokens	per model	A larger <code>max_tokens</code> is clamped down, not refused.
Upstream timeout	10 minutes	<code>504</code> . Set your own client timeout above this.
Output rate	per model, per key kind	A company key's stream may be paced to a tokens-per-second ceiling. Private keys are never paced.

There is no message-count limit of its own: a conversation is bounded by the model's context length and by the body size above. CORS is configured per relay; preflight is answered on every public path.

10. Copy-paste examples

Streaming, curl

```
curl -N https://<your-relay-host>/v1/chat/completions \
-H "Authorization: Bearer Kunci-Zeiko-XXXXXXXXXXXXXXXXXXXXXXXXXXXX" \
-H "Content-Type: application/json" \
-H "X-User-Id: tenant-42:user-9f3c1ab7" \
-d '{
  "model": "Wissangeni-512B-V1",
  "messages": [{"role": "user", "content": "Hello!"}],
  "stream": true
}'
```

Non-streaming, OpenAI Python SDK

```
from openai import OpenAI

client = OpenAI(
    base_url="https://<your-relay-host>/v1",
    api_key="Kunci-Zeiko-XXXXXXXXXXXXXXXXXXXXXXXXXXXX",
)

reply = client.chat.completions.create(
    model="Wissangeni-512B-V1",
    messages=[{"role": "user", "content": "Hello!"}],
    user="tenant-42:user-9f3c1ab7", # KV-cache isolation – send it every time
)

print(reply.id) # == the X-Relay-Request-Id header
print(reply.choices[0].message.content)
print(reply.usage.prompt_tokens, reply.usage.completion_tokens)
print(reply.usage.model_extra.get("cost"))
```

Streaming, OpenAI Python SDK

```
stream = client.chat.completions.create(
    model="Wissangeni-512B-V1",
    messages=[{"role": "user", "content": "Write a haiku."}],
    user="tenant-42:user-9f3c1ab7",
    stream=True,
)

usage = None
for chunk in stream:
    if chunk.usage is not None: # the final frame: choices is empty here
        usage = chunk.usage
        continue
    if not chunk.choices: # guard – do not assume choices[0] exists
        continue
    piece = chunk.choices[0].delta.content
    if piece:
        print(piece, end="", flush=True)

print("\n", usage)
```

Streaming, Node with fetch

```
const res = await fetch("https://<your-relay-host>/v1/chat/completions", {
  method: "POST",
  headers: {
```



```

    "Authorization": "Bearer Kunci-Zeiko-XXXXXXXXXXXXXXXXXXXXXXXXXXXX",
    "Content-Type": "application/json",
    "X-User-Id": "tenant-42:user-9f3c1ab7",
  },
  body: JSON.stringify({
    model: "Wissangeni-512B-V1",
    messages: [{ role: "user", content: "Hello!" }],
    stream: true,
  }),
});

const reader = res.body.getReader();
const decoder = new TextDecoder();
let buf = "";

while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  buf += decoder.decode(value, { stream: true });

  // Frames are separated by a blank line; tolerate CRLF from proxies.
  let m;
  while ((m = /\r?\n\r?\n/.exec(buf)) !== null) {
    const frame = buf.slice(0, m.index);
    buf = buf.slice(m.index + m[0].length);

    for (const line of frame.split(/\r?\n/)) {
      if (line.startsWith(":")) continue; // keep-alive comment
      if (!line.startsWith("data:")) continue;
      const data = line.slice(5).trim();
      if (data === "[DONE]") continue; // terminator, not JSON

      const json = JSON.parse(data);
      if (json.error) { console.error(json.error.message); continue; }
      if (json.usage) { console.log("\ncost:", json.usage.cost); }
      const piece = json.choices?.[0]?.delta?.content;
      if (piece) process.stdout.write(piece);
    }
  }
}

```

Listing models

```

curl https://<your-relay-host>/v1/models \
  -H "Authorization: Bearer Kunci-Zeiko-XXXXXXXXXXXXXXXXXXXXXXXXXXXX"

curl https://<your-relay-host>/v1/models/Wissangeni-512B-V1 \
  -H "Authorization: Bearer Kunci-Zeiko-XXXXXXXXXXXXXXXXXXXXXXXXXXXX"

```

Getting help. Every request has a UUIDv4 that appears as `id` in the body and as the `X-Relay-Request-Id` response header. Log it. A support question that quotes one can be traced to the exact request row — timing, token counts, the backend that served it and what it cost. A question without one usually cannot.